

Melhorando o Tratamento de Erros da API

Uma API profissional deve ser previsível não apenas quando tudo funciona, mas também quando algo falha. Neste material, você aprenderá a tratar erros de forma padronizada, clara e eficiente com NestJS.

</> NESTJS

TRATAMENTO DE ERROS

API REST

Objetivos de Aprendizagem

Ao final deste material, você será capaz de dominar os seguintes conceitos e práticas:

Exceções Nativas

Utilizar corretamente as exceções nativas do NestJS para cada cenário de erro.

Tipos de Erro

Diferenciar erros de validação, autenticação, autorização e regras de negócio.

Padronização

Padronizar mensagens retornadas pela API para consumo consistente.

Filtro Global

Implementar um filtro global de exceções e produzir respostas profissionais.

Por que Tratar Erros Corretamente?

Uma API não é composta apenas por respostas de sucesso. Na prática, muitos cenários podem falhar e precisam ser comunicados de forma clara.

Cenários Comuns de Falha

- Usuário não encontrado
- E-mail já cadastrado
- Senha inválida
- Token ausente ou expirado
- Dados obrigatórios não informados
- Erros inesperados do servidor

O Problema das Mensagens Genéricas

Quando esses problemas não são tratados adequadamente, o consumidor da API recebe mensagens pouco úteis como:

```
{ "message": "Internal server error" }
```

ou ainda pior:

```
{ "error": "Something went wrong" }
```

Mensagens genéricas dificultam a identificação e correção dos problemas tanto para o frontend quanto para o time de desenvolvimento.

O Papel dos Códigos HTTP

O código HTTP deve representar corretamente o problema ocorrido. Retornar códigos incorretos gera comportamentos inesperados no frontend e dificulta o diagnóstico de falhas.

Código	Significado	Quando Usar
400	Requisição inválida	Dados malformados ou ausentes
401	Não autenticado	Token ausente ou inválido
403	Sem permissão	Usuário autenticado mas sem acesso
404	Recurso não encontrado	Entidade inexistente no banco
409	Conflito de negócio	E-mail duplicado, regra violada
500	Erro interno	Falhas inesperadas do servidor

Usar o código HTTP correto é a primeira etapa para uma API bem projetada. O frontend depende desses códigos para tomar decisões de exibição e fluxo.

Exceções Nativas do NestJS

O NestJS já fornece diversas exceções prontas para uso. Basta importá-las do pacote `@nestjs/common` e utilizá-las nos lugares corretos da aplicação.

Importação

```
import {  
  BadRequestException,  
  UnauthorizedException,  
  ForbiddenException,  
  NotFoundException,  
  ConflictException,  
} from '@nestjs/common';
```

Exceções Disponíveis

NotFoundException

404 — Recurso não encontrado

ConflictException

409 — Conflito de dados

UnauthorizedException

401 — Não autenticado

ForbiddenException

403 — Sem permissão

Tratando Erros Comuns na Prática

Veja como aplicar as exceções nativas do NestJS nos cenários mais frequentes de uma API REST.

Recurso Inexistente — 404

```
const user = await this.usersRepository.findById(id);

if (!user) {
  throw new NotFoundException(
    'Usuário não encontrado'
  );
}
```

Resposta gerada:

```
{ "statusCode": 404, "message": "Usuário não encontrado", "error": "Not Found" }
```

E-mail Duplicado — 409

```
const existingUser =
  await this.usersRepository.findByEmail(email);

if (existingUser) {
  throw new ConflictException(
    'E-mail já cadastrado'
  );
}
```

E-mail Duplicado — 409 - **continuação**

Resposta gerada:

```
{ "statusCode": 409, "message": "E-mail já cadastrado", "error": "Conflict" }
```

Credenciais Inválidas — 401

```
if (!passwordIsValid) {  
    throw new UnauthorizedException(  
        'Credenciais inválidas'  
    );  
}
```

Resposta gerada:

```
{ "statusCode": 401, "message": "Credenciais inválidas", "error": "Unauthorized" }
```

Falta de Permissão — 403

```
if (user.role !== 'admin') {  
    throw new ForbiddenException(  
        'Acesso não permitido'  
    );  
}
```

Resposta gerada:

```
{ "statusCode": 403, "message": "Acesso não permitido", "error": "Forbidden" }
```

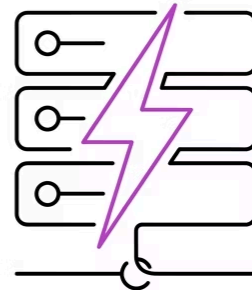
Separando Erros Técnicos de Erros de Negócio

Nem todo erro é um erro técnico. Essa distinção é fundamental para manter a aplicação organizada, facilitar a manutenção e comunicar problemas de forma adequada ao consumidor da API.



ERROS DE NEGÓCIO

E-mail já cadastrado
Produto sem estoque
Turma lotada
Disciplina encerrada



ERROS TÉCNICOS

Banco indisponível
Falha de rede
Timeout
Erro inesperado

Erros de negócio devem ser comunicados com mensagens claras e amigáveis. Erros técnicos devem ser registrados em logs internos e nunca expostos diretamente ao cliente.

Evitando Mensagens Internas e Criando um Padrão Corporativo

✗ Nunca Exponha Detalhes Internos

O cliente não precisa conhecer a estrutura interna do banco de dados ou do servidor.

```
{  
  "message": "duplicate key value violates  
  unique constraint users_email_key"  
}
```

Mensagens internas expõem a arquitetura da aplicação e representam um risco de segurança.

✓ Prefira Mensagens Claras

```
{ "message": "E-mail já cadastrado" }
```

Padrão Corporativo Recomendado

Uma API profissional normalmente utiliza um formato único e consistente para todas as respostas de erro:

```
{  
  "success": false,  
  "statusCode": 404,  
  "message": "Usuário não encontrado",  
  "timestamp": "2026-06-07T15:30:00Z",  
  "path": "/users/10"  
}
```

Esse formato facilita o tratamento de erros no frontend, permitindo exibição de mensagens amigáveis, notificações automáticas e registro de logs.

Criando e Registrando o Filtro Global de Exceções

O filtro global garante que **toda exceção** da aplicação siga o mesmo formato padronizado, sem precisar repetir lógica em cada controller.

Arquivo do Filtro

```
common/filters/http-exception.filter.ts
```

```
import {  
  ExceptionFilter,  
  Catch,  
  ArgumentsHost,  
  HttpException,  
} from '@nestjs/common';  
  
@Catch(HttpException)  
export class HttpExceptionFilter  
  implements ExceptionFilter {
```

```

catch(
  exception: HttpException,
  host: ArgumentsHost,
){
  const ctx = host.switchToHttp();
  const response = ctx.getResponse();
  const request = ctx.getRequest();

  response.status(
    exception.getStatus(),
  ).json({
    success: false,
    statusCode: exception.getStatus(),
    timestamp: new Date().toISOString(),
    path: request.url,
    message: exception.message,
  });
}
}

```

Registrando no main.ts

```

app.useGlobalFilters(
  new HttpExceptionFilter(),
);

```

Após registrar o filtro global no `main.ts`, toda exceção lançada na aplicação seguirá automaticamente o mesmo formato de resposta.

Estrutura de Pastas Recomendada

```

src
├── common
│   └── filters
│       └── http-exception.filter.ts
├── auth
├── users
├── products
└── ...

```

Essa organização permite reutilização do filtro em toda a aplicação e facilita a localização do código.

Benefícios para o Frontend

Quando os erros seguem um padrão consistente, o time de frontend consegue trabalhar de forma muito mais eficiente e autônoma.



Mensagens Amigáveis

Exibir mensagens claras e contextuais para o usuário final, sem precisar interpretar erros técnicos.



Destaque de Campos

Identificar e destacar automaticamente os campos com erro nos formulários da interface.



Notificações Automáticas

Criar notificações automáticas baseadas no `statusCode` e na `message` retornados.



Registro de Logs

Registrar logs estruturados com `timestamp` e `path` para rastreamento de problemas.



Cenários Específicos

Tratar cenários específicos como 401 (redirecionar para login) e 403 (exibir tela de acesso negado).

Com um padrão único, o frontend não precisa interpretar respostas diferentes para cada endpoint — reduzindo bugs e acelerando o desenvolvimento.

Checklist de Validação e Conclusão

Antes de concluir este material, verifique se todos os itens foram implementados corretamente na sua aplicação:

Itens de Implementação

- ✓ `NotFoundException` implementada
- ✓ `ConflictException` implementada
- ✓ `UnauthorizedException` implementada
- ✓ `ForbiddenException` implementada
- ✓ Mensagens internas removidas
- ✓ Respostas padronizadas
- ✓ Filtro global criado
- ✓ Filtro global registrado no `main.ts`
- ✓ Testes realizados em diferentes cenários de erro

Conclusão

Uma API profissional deve ser previsível não apenas quando tudo funciona, mas também quando algo falha.

O tratamento adequado de erros **reduz o tempo de diagnóstico**, facilita a integração com aplicações frontend e melhora significativamente a experiência dos desenvolvedores que consomem a API.

Ao padronizar respostas e utilizar corretamente as exceções do NestJS, criamos uma base sólida para sistemas mais **robustos, seguros e fáceis de manter**.

6

Tipos de Exceção

Exceções nativas do NestJS cobertas

1

Filtro Global

Para padronizar toda a aplicação

9

Itens no Checklist

Para validar sua implementação

